SARDAR PATEL INSTITUTE OF TECHNOLOGY · DEPARTMENT OF COMPUTER ENGINEERING

Autonomous Institute Affiliated to University of Mumbai

Functional and Non-Functional Testing

Phase IV · Deliverable P2

EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh

Course	T.E. Semester V · Mini Project – I · Academic year 2026–27
Group	Group 20
Team	Twissha Shah · 2024300231
	Miti Shah · 2024300230
	Tanvi Sheth · 2024300239
Guide	Prof. Anuj Tawari
Version	1.0 · 28 October 2026
Code tested	https://github.com/tanvisheth1234-wq/miniproject branch develop_6 · commit 8eb3bf8

Contents

1 Introduction 3

1.1 Purpose 3

1.2 Scope 3

1.3 Reference documents 3

2 Test strategy 4

2.1 Test levels 4

2.2 Functional versus non-functional 4

2.3 Entry and exit criteria 4

3 Test environment and tools 5

3.1 Environment 5

3.2 Test data 5

3.3 How to reproduce every result 5

4 Execution summary 6

4.1 Automated suite, latest run 6

4.2 Overall verdict 6

5 Functional testing 7

5.1 Approach 7

5.2 Traceability summary 7

5.3 Requirement-by-requirement results 7

5.4 Representative test cases 11

5.5 Device-screen functional tests (browser) 12

5.6 Simulation-lab website tests (browser) 13

5.7 API tests 13

5.8 Boundary, invalid-input and concurrency tests 14

6 Non-functional testing 15

6.1 Validation against the ten target metrics 15

6.2 Performance 15

6.3 Reliability under packet loss 16

6.4 Scalability 16

6.5 Routing quality, recovery and availability 16

6.6 Security testing 17

6.7 Resource use 17

6.8 Usability, accessibility and compatibility 17

6.9 Maintainability 18

7 Defects found and resolved 19

8 Limitations and risks 20

9 Conclusion and sign-off 21

A Appendix A: all 332 automated test cases 22

1. Introduction

This document is the Phase IV, deliverable **P2** submission: the functional and non-functional testing of **EcoMesh**, a distributed peer-to-peer communication mesh for disaster relief. In EcoMesh every device is both an endpoint and a relay, so a message from someone with no signal hops from device to device until it reaches a gateway or a rescue team, with no server and no internet.

1.1 Purpose

To show, with repeatable evidence, that the system does what its Software Requirements Specification (SRS) says (**functional testing**) and that it does so quickly enough, reliably enough, securely enough and at the scale the project targets (**non-functional testing**). Where a result falls short of its target the document says so and says why.

1.2 Scope

In scope	Out of scope (stated openly)
The nine engine modules and the node that joins them; the device app (survivor, gateway and rescue screens, the one app) and its server API; the simulation lab (REST and WebSocket API and React website); the simulator and its seven scenarios; a five-process real-socket deployment.	Real radio hardware (Bluetooth LE or Wi-Fi Direct); a native mobile application; a timed usability study with participants; browsers other than Chrome; printing and installation of the PWA on physical phones.

1.3 Reference documents

- Software Requirements Specification: 68 functional requirements in 10 features (identifiers such as `RTE-4`).
- Project report, Chapter 9 (Testing and Validation) and Chapter 10 (Results), which this document extends and must not contradict.
- Master project plan, section 16: the ten target metrics (K1 to K10) used in Section 6.
- Source repository: https://github.com/tanvisheth1234-wq/miniproject, branch `develop_6`.

2. Test strategy

2.1 Test levels

Level	What is exercised	Tooling	When / how often
Unit	One module in isolation: packet, discovery, routing, reliability, crypto, store, relay	pytest	Developer, on every change
Integration	Modules together inside one node; several nodes over the simulated radio and over real TCP	pytest, pytest-asyncio	Developer, on every change
System (API)	Front end to back end to engine: REST and WebSocket against a live mesh	FastAPI test client, httpx	Developer, on every change
System (device screens)	The one app in a real browser against five live device processes: SOS with need and location, I am safe, messages, replies, filters, history, CSV export, gateway off and on	Headless Chrome, puppeteer-core	Scripted run (Section 5.5)
System (simulation lab)	The lab website driven as a user would: menus, the long-press SOS, the six demo beats	Headless Chrome, puppeteer-core	Scripted run (Section 5.6)
Simulation	Whole meshes of 5 to 65 nodes with seeded loss, latency and scripted failures, on a virtual clock	sim/scenarios.py, api/vtime.py	20 to 50 seeded runs per scenario
Live devices	Five real processes with their own sockets: latency, failover, store-and-forward	story/ + live_test.py	Repeated trials

2.2 Functional versus non-functional

	Functional testing	Non-functional testing
Question	Does each feature behave as the SRS requires?	How well does it behave: how fast, how reliably, how securely, at what scale?
Basis	The 68 numbered functional requirements, each with an acceptance criterion (the tests that must pass)	Ten key performance indicators (K1 to K10) fixed in the project plan before measuring
Pass rule	Every mapped test passes in a clean run	The measured value meets its target; a miss is reported as a miss
Section	5	6

2.3 Entry and exit criteria

- **Entry:** the code under test is on the phase branch, builds, and the suite can be started with one command.
- **Exit (functional):** every requirement is either verified by passing tests, marked partial with the reason, or listed as verified by inspection or a manual check.
- **Exit (non-functional):** each of the ten KPIs has a measured value from stored raw data; any KPI not met or met only on average is explained.
- **Rule we held to:** a failing or flaky test is recorded in Section 7, never deleted or hidden.

3. Test environment and tools

3.1 Environment

Item	Value
Machine / OS	Windows 11 Home, one laptop (all processes on one host)
Language and runtime	Python 3.11.9 (the code base targets Python 3.10 or newer); Node.js 22 for the UI build and the browser test
Test framework	pytest 9.1.1 with pytest-asyncio 1.4.0; FastAPI 0.139 test client with httpx 0.28
Browser	Google Chrome (headless) controlled by puppeteer-core; desktop viewports and phone viewports of 390 and 320 px wide
Simulator	In-process `SimulatedTransport` with seeded loss and latency; a virtual-time event loop so the specified 3 s HELLO, 10 s timeout and 4 s ACK timer values run in milliseconds
Live deployment	Five real processes (`python -m story.local`) with TCP between them and a topology file that emulates radio range
Code under test	branch `develop_6`, commit `8eb3bf8`, tested on 28 October 2026

3.2 Test data

There is no external dataset. Inputs are generated: message text and priorities from the tests and the website; random byte strings and bit-flipped packets for fuzzing (seeded, so a failure is reproducible); and seeded topologies (grid, chain, two-path) for the simulator, with the seed recorded in every result file.

3.3 How to reproduce every result

```
pip install -r requirements.txt
python -m pytest -q --junitxml=docs/testing/results/pytest_latest.xml     # 332 automated tests
node docs/testing/device_walkthrough.cjs                                    # starts the five devices
itself; checks of the device screens
python -m api.server                                                        # terminal 1: the
simulation lab website and API
node docs/testing/ui_walkthrough.cjs                                        # terminal 2: checks of
the lab website
python -m sim.scenarios --scenario collapse_40 --runs 50                   # one simulator scenario
python docs/report/evidence/collect.py all                                  # all simulator
measurements
python docs/report/evidence/live_test.py                                    # five-process live trials
python docs/testing/generate_testing_document.py                            # rebuild this document
```

4. Execution summary

332/332	**18/18**	**68**	**8 of 10**
automated tests passed in the latest run (2026-10-28)	browser checks passed on the device screens	functional requirements, each traced to evidence	target metrics fully met; 2 partly met, explained

4.1 Automated suite, latest run

Run on 2026-10-28: **332 passed, 0 failed** in 25.9 s. The suite was also run three times in a row earlier for the report (run 1: 332 passed, 1 warning, run 2: 332 passed, 1 warning, run 3: 332 passed, 1 warning); 3 of the 3 were fully green. Times below are the sum of the tests in each file.

Test file	Tests	Passed	Time (s)	What it covers
`test_packet.py`	47	**47**	0.0	Wire format: 52-byte header, path, payload, signature, retransmission counter, malformed input
`test_link_state.py`	33	**33**	0.1	Link-state encoding, gossip, re-flooding, report replacement, convergence
`test_node.py`	28	**28**	4.2	Whole node: discovery + relay + routing + reliability + security + logging together
`test_store.py`	27	**27**	0.0	SQLite message, event and peer log and its queries
`test_ack_retry.py`	20	**20**	0.0	ACK tracker, retry limits, store-and-forward bounds, eviction and expiry
`test_crypto.py`	19	**19**	0.0	AES-256-GCM encryption and Ed25519 signatures, wrong key, tampering
`test_reliability.py`	18	**18**	0.0	Duplicate suppression and the priority queue
`test_relay.py`	18	**18**	0.2	Forwarding, TTL, loop detection, priority at a relay, dedup rules
`test_transport_sim.py`	15	**15**	0.1	Simulated radio: links, loss, latency, delivery
`test_discovery.py`	15	**15**	1.1	HELLO beacons, neighbour table, death detection, range emulation
`test_routing.py`	14	**14**	0.0	Cost function and weighted path selection
`test_dijkstra.py`	14	**14**	0.0	Shortest path, RESCUE__ resolution, route cache
`test_link_quality.py`	12	**12**	0.0	Link quality measured from HELLO reception; hop-count routing mode
`test_api.py`	12	**12**	4.4	REST and WebSocket API against a live mesh (front end to engine)
`test_robustness.py`	11	**11**	3.8	Fuzzing, boundary input, concurrency and stress
`test_lan_transport.py`	6	**6**	3.0	Real TCP transport between processes, range emulation, three-node mesh
`test_sim_runner.py`	5	**5**	0.3	Simulator runner and topologies
`test_ack_multihop.py`	3	**3**	1.5	ACK over several hops, lost-ACK re-send, retry through relays
`test_device_api.py`	12	**12**	6.0	The device server: events, send, history filters, structured SOS, I am safe, CSV export, rescue reply
`test_conftest_helpers.py`	3	**3**	0.1	The wait_until polling helpers used by the timing-sensitive tests
Total	**332**	**332**	**24.8**	

4.2 Overall verdict

Functional: 64 of the 68 requirements are fully verified by automated tests (52 by tests alone, 12 by tests plus a check of the screen). 0 are only partly implemented, 2 were verified by code inspection and 2 by a manual check. **Non-functional:** 8 of the 10 target metrics are fully met; the other 2 are met only partly, and Section 6 gives the measured numbers and the reasons.

5. Functional testing

5.1 Approach

The SRS numbers every requirement (feature prefix and index, for example `PRI-3`). Each requirement's **acceptance criterion** is the set of tests that must pass for it, recorded in `traceability.py`. A requirement is then verified by one of five methods: automated tests alone; automated tests plus a manual look at the screen; code inspection; a manual check; or it is only partly implemented. The table in 5.3 lists all 68, with the test that verifies each and today's result.

5.2 Traceability summary

Feature	Prefix	Requirements	Verified by tests	Partial	Inspection or manual
Peer discovery	`DIS`	7	7	0	0
Message composition and transmission	`MSG`	7	7	0	0
Multi-hop forwarding	`FWD`	7	7	0	0
Adaptive routing	`RTE`	8	8	0	0
Priority handling	`PRI`	6	5	0	1
Reliability	`REL`	8	8	0	0
Store-and-forward	`SNF`	6	5	0	1
Message security	`SEC`	7	6	0	1
Logging and history	`LOG`	6	5	0	1
Rescue dashboard	`DSH`	6	6	0	0
Total		**68**	**64**	**0**	**4**

5.3 Requirement-by-requirement results

"Pass" means every mapped test passed in the latest run. For requirements with several tests only the first is shown, with the number of others. Notes under a requirement explain anything that is not a clean pass.

ID	The system shall…	Method	Verifying test	Result
5.3.1 Peer discovery *Input: HELLO beacons (role, public key) from neighbours; silence from a known neighbour. Output: Peer-up and peer-down events and link quality passed to routing.*				
DIS-1	transmit a discovery beacon at a fixed, configurable interval.	Automated	`test_hello_is_broadcast_on_start`	**Pass**
DIS-2	keep a neighbour table with node id, address, role, public key, last-seen time and measured link quality.	Automated	`test_update_reports_new_neighbour_once` *(+3 more)*	**Pass**
DIS-3	add a previously unknown node to the table on receipt of a valid beacon.	Automated	`test_neighbour_added_on_hello_received` *(+1 more)*	**Pass**
DIS-4	declare a neighbour unreachable after a configurable period of silence.	Automated	`test_neighbour_marked_dead_after_timeout` *(+1 more)*	**Pass**
DIS-5	compute link quality as the proportion of expected beacons received over a sliding window.	Automated	`test_perfect_reception_scores_one` *(+2 more)*	**Pass**
DIS-6	operate without manual configuration of peer addresses.	Automated	`test_message_to_discovered_neighbour_delivers_without_manual_next_hop` *(+1 more)*	**Pass**
DIS-7	support an optional configured seed list of peers where multicast is unavailable.	Automated	`test_visible_neighbours_filters_unlisted_senders` *(+1 more)*	**Pass**

ID	The system shall…	Method	Verifying test	Result
5.3.2 Message composition and transmission *Input: Text and destination, or the SOS control (sustained press). Output: DATA packet sent; delivery state shown to the user.*				
MSG-1	let a user compose and send a text message to a named node.	Automated + UI	`test_send_with_ack_delivers_and_gets_acknowledged` *(+3 more)*	**Pass**
MSG-2	let a user address a message to the rescue service rather than a specific node.	Automated	`test_rescue_dst_routes_to_gateway_across_multiple_hops` *(+1 more)*	**Pass**
MSG-3	provide a distinct SOS control that requires a sustained press.	Automated + UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`	**Pass**
MSG-4	assign a globally unique identifier to every message it originates.	Automated	`test_msg_id_defaults_to_uuid4_and_round_trips` *(+1 more)*	**Pass**
MSG-5	record the creation time of every message it originates.	Automated	`test_timestamp_is_uint64_milliseconds` *(+1 more)*	**Pass**
MSG-6	display the current delivery state of each originated message.	Automated + UI	`test_store_logs_sent_message_and_delivery_lifecycle` *(+1 more)*	**Pass**
MSG-7	display the number of intermediate nodes traversed by a delivered message.	Automated + UI	`test_update_message_route_records_hops_and_path` *(+1 more)*	**Pass**
5.3.3 Multi-hop forwarding *Input: DATA or ACK packet from a neighbour. Output: Packet forwarded, or dropped with a logged reason.*				
FWD-1	forward packets not addressed to it towards the destination.	Automated	`test_two_hop_relay_a_to_b_to_c` *(+1 more)*	**Pass**
FWD-2	decrement the remaining hop allowance of every packet it forwards.	Automated	`test_on_forward_hook_called_with_next_hop`	**Pass**
FWD-3	discard any packet whose hop allowance has reached zero.	Automated	`test_drops_when_ttl_expired`	**Pass**
FWD-4	append its own identifier to the traversal path of every forwarded packet.	Automated	`test_on_forward_hook_called_with_next_hop` *(+1 more)*	**Pass**
FWD-5	discard any packet whose traversal path already contains its own identifier.	Automated	`test_drops_on_loop_detected`	**Pass**
FWD-6	validate the structure of a packet before any forwarding decision.	Automated	`test_bad_magic_rejected` *(+3 more)*	**Pass**
FWD-7	be unable to read the payload of a packet it merely forwards.	Automated	`test_relay_never_sees_plaintext_payload`	**Pass**
5.3.4 Adaptive routing *Input: Link-state reports; a destination. Output: Least-cost next hop; routes recomputed on topology change.*				
RTE-1	periodically transmit its own neighbour list to adjacent nodes.	Automated	`test_send_report_reaches_each_neighbour` *(+1 more)*	**Pass**
RTE-2	construct a graph of the network from received link-state reports.	Automated	`test_receiving_report_merges_reported_edges_into_graph` *(+1 more)*	**Pass**
RTE-3	compute least-cost paths using Dijkstra's algorithm over the weighted graph.	Automated	`test_shortest_path_multi_hop_chain` *(+1 more)*	**Pass**
RTE-4	derive link cost from a base hop penalty, measured link quality and recent delivery failures.	Automated	`test_edge_cost_matches_section_8_2_formula` *(+1 more)*	**Pass**

ID	The system shall…	Method	Verifying test	Result
RTE-5	recompute routes whenever the topology changes.	Automated	`test_record_failure_invalidates_cached_routes` *(+2 more)*	**Pass**
RTE-6	treat any node advertising the gateway or rescue role as a valid terminal for rescue-service messages.	Automated	`test_rescue_dst_resolves_to_nearest_gateway` *(+2 more)*	**Pass**
RTE-7	select the least-cost terminal when several are reachable.	Automated	`test_rescue_dst_picks_cheapest_of_multiple_gateways`	**Pass**
RTE-8	converge on a consistent topology view within a bounded interval after a change.	Automated	`test_report_reflooded_reaches_a_node_two_hops_away` *(+1 more)*	**Pass**
5.3.5 Priority handling *Input: Packets of different priority queued together. Output: SOS transmitted before routine traffic.*				
PRI-1	support three priority levels: SOS, NORMAL, STATUS.	Automated	`test_all_three_priorities_ordered_correctly` *(+1 more)*	**Pass**
PRI-2	keep outgoing messages in a priority queue rather than a FIFO queue.	Automated	`test_sos_pops_before_normal_regardless_of_insertion_order`	**Pass**
PRI-3	apply priority ordering at every hop, not only at the originator.	Automated	`test_sos_forwarded_before_already_queued_normal` *(+1 more)*	**Pass**
PRI-4	order messages of equal priority by creation time, oldest first.	Automated	`test_ties_broken_by_oldest_timestamp_first`	**Pass**
PRI-5	assign SOS priority to every message generated by the SOS control.	Automated + UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`	**Pass**
PRI-6	display the priority level of each message in the interface.	Manual only	*Priority badges are shown on the screens (manual check); no automated UI test.*	**Manual check**
5.3.6 Reliability *Input: Delivered or unacknowledged packets; duplicates; missing beacons. Output: ACK returned; bounded retries; duplicates dropped; failed neighbours removed.*				
REL-1	carry a hop allowance in every packet and decrement it at each hop.	Automated	`test_drops_when_ttl_expired` *(+2 more)*	**Pass**
REL-2	remember recently observed message identifiers for a configurable period.	Automated	`test_entry_expires_after_window` *(+1 more)*	**Pass**
REL-3	discard any packet whose identifier is already remembered.	Automated	`test_duplicate_delivery_is_dropped` *(+2 more)*	**Pass**
REL-4	transmit an acknowledgement on successful delivery.	Automated	`test_send_with_ack_delivers_and_gets_acknowledged` *(+1 more)*	**Pass**
REL-5	retransmit an unacknowledged message after a configurable timeout.	Automated	`test_timeout_triggers_retry_and_resets_timer` *(+1 more)*	**Pass**
REL-6	recompute the route before each retransmission.	Automated	`test_record_failure_updates_graph_immediately_not_just_on_next_report` *(+1 more)*	**Pass**
REL-7	abandon delivery after a configurable number of failed attempts and mark the message accordingly.	Automated	`test_exhausting_max_retries_reports_failed_and_stops_tracking` *(+1 more)*	**Pass**
REL-8	detect node failure through absence of discovery beacons and update the graph.	Automated	`test_route_forgotten_when_neighbour_dies` *(+2 more)*	**Pass**

ID	The system shall…	Method	Verifying test	Result
5.3.7 Store-and-forward *Input: A message with no route; a new route or neighbour. Output: Message delivered when a path appears, or expired and logged.*				
SNF-1	retain a message when route computation yields no path.	Automated	`test_send_without_route_queues_to_store_forward`	**Pass**
SNF-2	periodically re-attempt delivery of retained messages.	Automated	`test_store_forward_flushes_once_a_route_appears`	**Pass**
SNF-3	attempt delivery of retained messages immediately on detecting a new neighbour.	Automated	`test_held_message_is_sent_the_moment_its_neighbour_appears` *(+1 more)*	**Pass**
SNF-4	bound the retention queue by both message count and retention period.	Automated	`test_enqueue_over_the_bound_evicts_lowest_priority` *(+2 more)*	**Pass**
SNF-5	discard the lowest-priority retained messages first when the bound is reached.	Automated	`test_enqueue_over_the_bound_evicts_lowest_priority` *(+2 more)*	**Pass**
SNF-6	report the number of retained messages in the interface.	Manual only	*The held-message count is shown on the screens (manual check); no automated UI test.*	**Manual check**
5.3.8 Message security *Input: Payload and packet identity fields. Output: Tampered or forged packets rejected.*				
SEC-1	encrypt message payloads using authenticated symmetric encryption.	Automated	`test_encrypt_decrypt_round_trip` *(+2 more)*	**Pass**
SEC-2	ensure intermediate nodes cannot recover the plaintext of messages they forward.	Automated	`test_relay_never_sees_plaintext_payload`	**Pass**
SEC-3	sign the packet header and traversal path with a digital signature scheme.	Automated	`test_signed_message_verifies_and_delivers_one_hop` *(+2 more)*	**Pass**
SEC-4	verify the signature of every received packet before delivery.	Automated	`test_forged_signature_is_rejected_not_delivered` *(+1 more)*	**Pass**
SEC-5	discard any packet failing signature verification or authentication.	Automated	`test_forged_signature_is_rejected_not_delivered` *(+2 more)*	**Pass**
SEC-6	distribute node public keys through discovery beacons.	Automated	`test_hello_includes_public_key_when_configured` *(+1 more)*	**Pass**
SEC-7	use established cryptographic libraries and not implement primitives directly.	Inspection	*Code inspection: crypto.py only calls the cryptography library (AESGCM, Ed25519).*	**Inspection**
5.3.9 Logging and history *Input: Every message and network event. Output: History and delivery statistics available by query.*				
LOG-1	maintain a persistent local record of messages sent, received and forwarded.	Automated	`test_store_logs_forward_on_middle_node` *(+1 more)*	**Pass**
LOG-2	record status, creation time, resolution time, hop count and path of each message.	Automated	`test_record_message_stores_path_as_comma_separated` *(+2 more)*	**Pass**
LOG-3	record network events: node arrival and failure, route change, duplicate suppression, hop-allowance expiry, retransmission, retention.	Automated	`test_store_logs_peer_up_and_down` *(+2 more)*	**Pass**

ID	The system shall…	Method	Verifying test	Result
LOG-4	store plaintext content only where the node is the originator or destination.	Inspection	`test_store_logs_forward_on_middle_node` *Forwarded rows have body NULL (the sample rows in report Section 4.6); asserted for the relay's log, otherwise by inspection.*	**Inspection**
LOG-5	let the user view and filter the local message history.	Automated + UI	`test_history_filters_by_priority_and_direction` *(+2 more)*	**Pass**
LOG-6	support extraction of delivery statistics from the stored records.	Automated	`test_list_and_count_messages_filter_by_direction_and_status` *(+1 more)*	**Pass**
5.3.10 Rescue dashboard *Input: Node state and events. Output: Live graph; SOS queue by priority then age; statistics.*				
DSH-1	render the mesh topology as a graph of nodes and active links.	Automated + UI	`test_snapshot_describes_a_converged_mesh`	**Pass**
DSH-2	update the topology view as nodes join and fail.	Automated + UI	`test_killing_a_node_reroutes_and_reviving_restores_it`	**Pass**
DSH-3	display outstanding emergency messages ordered by priority, then age.	Automated + UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`	**Pass**
DSH-4	display the originating node and traversal path of each received message.	Automated + UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered` *(+1 more)*	**Pass**
DSH-5	display aggregate statistics: delivery ratio, mean hop count, nodes online.	Automated + UI	`test_snapshot_describes_a_converged_mesh`	**Pass**
DSH-6	receive updates without the coordinator refreshing the view.	Automated + UI	`test_websocket_streams_a_hello_snapshot_then_events`	**Pass**

5.4 Representative test cases

Twelve cases chosen to cover every feature, written in the usual form: scenario and input, expected result, actual result and evidence, verdict.

ID	Req.	Scenario and input	Expected result	Actual result and evidence	Verdict
TC-01	RTE-3, 4	Short path over a lossy link; longer path over clean links	Dijkstra takes the clean path	As expected (test_dijkstra)	**Pass**
TC-02	RTE-8	A link-state report from a node two hops away	Re-flooded; the graph is complete beyond the neighbour	As expected (test_link_state)	**Pass**
TC-03	PRI-3	SOS reaches a relay that has routine packets queued	SOS is forwarded first	As expected (test_relay)	**Pass**
TC-04	REL-5, 6	ACK lost; the sender times out	Route recomputed, resent; destination sees the message once	As expected (test_ack_multihop)	**Pass**
TC-05	SNF-1, 2	No route; a route appears later	Held, then flushed and delivered	As expected; live: 4 of 4 outages flushed, 5 of 5 messages each time	**Pass**
TC-06	SNF-5	Retention queue full	Lowest priority evicted; an SOS is never displaced	As expected (test_ack_retry)	**Pass**
TC-07	SEC-5	A packet with a forged signature	Rejected and logged	As expected (test_node)	**Pass**
TC-08	FWD-6	3,000 random byte strings and 1,000 bit-flipped packets	Only parsed or PacketError; the node survives	One defect found and fixed (D4); now passes (test_robustness)	**Pass**
TC-09	DIS-7	Three nodes over real TCP, neighbour-restricted	Routes learned; message and ACK delivered	As expected (test_lan_transport)	**Pass**

ID	Req.	Scenario and input	Expected result	Actual result and evidence	Verdict
TC-10	K1	40 nodes, 20% loss, relay and gateway failures, 50 seeds	Delivery at least 90%	92.6% mean, worst run 78.6%	**Pass**
TC-11	K3	Kill the relay on the active path, 50 trials	Re-route within 10 s	Mean 8.1 s; 24 of 50 trials within 10 s; worst 10.25 s	**Partial**
TC-12	K4, K6	Congested radio; weighted vs hop-count on identical seeds	SOS at least 3 times faster; weighted delivers more	4.4 times; 99.2% vs 57.4%	**Pass**
TC-13	SNF-3	A message held for node C; C then appears	Sent at once, not after the 10 s flush interval	As expected (test_node); on devices: DT-15 and DT-16	**Pass**
TC-14	LOG-5	Message history filtered by priority and by direction	Only matching messages are listed; the CSV export has a header and rows	As expected (test_device_api); on screen: DT-12	**Pass**

5.5 Device-screen functional tests (browser)

The device screen is the one app and the demonstration, so it was tested the way a user meets it: a script starts five real device processes, opens the survivor phone, a second phone, the gateway and the rescue console in a real Chrome, presses the on-screen buttons and reads back what each page shows. The script is `docs/testing/device_walkthrough.cjs`; the results below are from its latest run (2026-10-27). Every outcome is produced by the real engine and read from the screen afterwards.

ID	Check	Evidence read from the page	Result
DT-01	Survivor screen loads and the app bar names the device	Trapped person / PHONE	**Pass**
DT-02	Rescue device shows the console, with its name in the app bar	Rescue team / RESCUE TEAM	**Pass**
DT-03	The mesh organises itself: the survivor is connected to the rescue team through phones in between	Trapped person→Phone 2→Gateway→Rescue team	**Pass**
DT-04	Releasing the SOS button after 0.4 s sends nothing	sent messages 0 -> 0	**Pass**
DT-05	A held SOS with need and location reaches the rescue console with the route it took	Trapped person > Phone 2 > Gateway > Rescue team	**Pass**
DT-06	The survivor screen confirms "The rescue team has your request"	Confirmed on the page	**Pass**
DT-07	"I am safe" from another phone appears under Marked safe, not among the requests	Phone 3 just now SAFE	**Pass**
DT-08	A typed message sent to the rescue team is delivered and listed on the console	Confirmed on the page	**Pass**
DT-09	Reply opens and a quick reply "Help is on the way" reaches the survivor	MESSAGE FROM RESCUE TEAM Help is on the way just now	**Pass**
DT-10	A typed reply survives the 0.7 s refresh (text and focus kept) and reaches the survivor	kept="Stay where you are, medics are..." focused=true	**Pass**
DT-11	Console filters: SOS only, Messages only, and by need	SOS=SOS; Messages=NORMAL; need=Trapped	**Pass**
DT-12	History filter shows only the chosen priority; the CSV button is shown and the export has a header and rows	history=1 SOS rows; csv 200 text/csv; charset=utf-8; header: time,from,priority,need,where,hops,route,message; rows: 3	**Pass**
DT-13	The "What this device is doing" panel lists live events in plain language	00:54:51Sent a message toward Phone 2 | 00:54:51A message arrived here | 00:54:47Sent a message toward Phone 2	**Pass**
DT-14	The gateway view says it is online and shows the messages it passed on	Gateway online This device links the mesh to the rescue team and passe	**Pass**
DT-15	With the gateway off, an SOS shows "Saved, waiting for a path"	lost route: true	**Pass**

ID	Check	Evidence read from the page	Result
DT-16	With the gateway back on, the held SOS is delivered and appears on the console	Confirmed on the page	**Pass**
DT-17	At 390 px and 320 px wide neither screen scrolls sideways	8101@390:0, 8101@320:0, 8105@390:0, 8105@320:0	**Pass**
DT-18	No script errors or console errors on any device screen during the run	none	**Pass**

5.6 Simulation-lab website tests (browser)

The React website is not part of the demonstration; it is the simulation lab that produced the experiments. It was tested the same way, with `docs/testing/ui_walkthrough.cjs` (latest run 2026-10-25) against the running API server.

ID	Check	Evidence read from the page	Result
UT-01	All 9 pages open from the top menu and render content	text length per page: 4188, 1384, 1403, 1219, 6035, 2263, 2248, 3464, 1955	**Pass**
UT-02	Releasing the SOS button early (0.5 s) does not send an SOS	SOS count 0 -> 0	**Pass**
UT-03	Holding the SOS button for 1.8 s sends an SOS that is delivered	last SOS status: delivered	**Pass**
UT-04	Beat 1: self-organisation (all nodes know the whole mesh)	All 5 nodes know all 5 nodes — no configuration was entered.	**Pass**
UT-05	Beat 2: an SOS sent last still arrives first	The SOS arrived first of 7, though it was sent last.	**Pass**
UT-06	Beat 3: kill the relay on the active path, next SOS reroutes	L2 is down. The new SOS still arrived via L1 → L3 → L4.	**Pass**
UT-07	Beat 4: gateway down, 5 SOS held in store-and-forward	5 SOS messages are being held at L1, undelivered but not lost.	**Pass**
UT-08	Beat 5: gateway back, all held SOS flush	5 of 5 held messages flushed through the returning gateway.	**Pass**
UT-09	Beat 6 opens the simulator page	http://127.0.0.1:8000/simulator	**Pass**
UT-10	All 5 mesh layouts load with the expected node counts	demo5:5, building12:13, collapse20:21, chain6:6, blank:0	**Pass**
UT-11	A switched-off node shows "This phone is off" and cannot send	Confirmed on the page	**Pass**
UT-12	The API refuses an SOS from a switched-off node (HTTP 409)	status 409	**Pass**
UT-13	Security drill: honest relay accepted, 3 attacks rejected	3 attacks rejected	**Pass**
UT-14	Simulator page: weighted routing beats hop-count on the same seeds	weighted 100% vs hop-count 54%	**Pass**
UT-15	At 390 px width no page scrolls sideways	overflow px per page: 0, 0, 0, 0, 0, 0, 0, 0, 0	**Pass**
UT-16	No console errors or script exceptions during the whole run	none	**Pass**

5.7 API tests

Twelve tests in `test_api.py` start a real mesh of five nodes and call the simulation-lab API through FastAPI's test client, including a live WebSocket. They cover the converged snapshot, an SOS reaching the rescue queue, hop distances in a node's view, route comparison, clear error codes for bad requests (404 unknown node, 409 node switched off, 422 empty text, 400 bad priority), killing and reviving a node, the event stream, the protocol inspector, the cryptography endpoints, the attack drill and resetting the mesh with encryption and signing on. Twelve more in `test_device_api.py` start real device processes and call the device API: the event feed, sending a message, rejecting bad sends, history filters, structured SOS, the safe check-in, CSV export and the rescue reply.

Test	Result
test_attack_drill_defends_every_attack	**Pass**
test_bad_requests_get_clear_errors	**Pass**
test_killing_a_node_reroutes_and_reviving_restores_it	**Pass**
test_node_view_reports_peers_with_hop_distance	**Pass**
test_presets_and_scenarios_are_listed	**Pass**
test_protocol_inspector_builds_and_parses_real_packets	**Pass**
test_reset_swaps_the_preset_and_security_options	**Pass**
test_route_compare_returns_both_routers	**Pass**
test_security_playground_round_trips_and_detects_tampering	**Pass**
test_snapshot_describes_a_converged_mesh	**Pass**
test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered	**Pass**
test_websocket_streams_a_hello_snapshot_then_events	**Pass**

5.8 Boundary, invalid-input and concurrency tests

Area	What is tested	Outcome
Boundary input	Empty and header-only input; bad magic value; truncated header, path, payload and signature; TTL out of range; payload over the 16-bit length; path too long; node id over 8 bytes or not ASCII; wrong-length signature; signed flag without a signature; wrong key size; ciphertext shorter than the nonce; an empty payload (valid); a send to an unknown destination (held, not an exception)	**Pass**
Fuzzing	3,000 random byte strings and 1,000 bit-flipped valid packets to the parser; 200 garbage buffers to a running node, which must still deliver a normal message afterwards	**Pass**
Concurrency	200 simultaneous sends over two hops with ACKs: all 200 delivered exactly once and every ACK returned. Two senders sending 50 SOS and 50 routine messages at once to one gateway: all 100 arrive intact	**Pass**
Replay	One packet replayed five times is delivered once and the duplicates are dropped	**Pass**
Real transport	TCP between processes: previous-hop identity, range emulation, a switched-off device neither sends nor hears, hostile bytes ignored, sending to a dead peer fails fast, a full three-node mesh delivers with ACK	**Pass**

6. Non-functional testing

6.1 Validation against the ten target metrics

The targets were fixed in the project plan before any measurement. Each value below comes from stored raw data in `docs/report/evidence/results/` and can be regenerated with the commands in Section 3.3.

ID	Metric	Target	Measured	Verdict
K1	Delivery ratio	at least 90% (40 nodes, 20% loss, failures injected)	92.6% mean over 50 runs (worst run 79%). Without failures, a 43-node grid at 20% loss: 96.2%	**Met**
K2	Convergence	15 s or less	mean 9.6 s in collapse_40; 47 of 50 runs within 15 s; worst 15.25 s; mean 17.7 s in the 35%-loss scenario. Values come in multiples of the 5 s gossip interval	**Partly met**
K3	Recovery after relay death	10 s or less	mean 8.1 s, median 10.2 s; 24 of 50 trials within 10 s. The other 26 took 10.2 to 10.25 s: the 10 s heartbeat timeout plus the 0.25 s sampling step	**Partly met**
K4	SOS latency advantage	at least 3 times faster	4.4 times (median routine latency over median SOS latency)	**Met**
K5	Store-and-forward flush	100% flushed	207 of 207 in the no-gateway scenario; 100% of live trials	**Met**
K6	Weighted vs hop-count routing	weighted is better	99.2% against 57.4% delivery on identical seeds	**Met**
K7	Scale	50 nodes within the targets	50 nodes at 5% loss: 100.0%; at 20% loss: 92.9%. Largest tested, 65 nodes: 98.0% (5% loss) and 83.1% (20% loss)	**Met**
K8	Screen freshness	2 s or less	API response median 16 ms (p95 31 ms); the screens refresh every 0.7 s	**Met**
K9	Correctness	100% of tests pass	332 of 332 passed in all 3 repeated runs	**Met**
K10	Security	all properties hold	a relay cannot read the payload; a forged signature, a tampered payload and a wrong key are all rejected	**Met**

Reading this table. K1, K4, K5, K6, K7, K8, K9, K10 are fully met. K2 (convergence) is only partly met: it holds in most runs but not at 35% loss, because convergence moves in multiples of the 5 s gossip interval. K3 (recovery after relay death) is only partly met: the mean is under 10 s, but recovery is defined by the 10 s heartbeat timer and measured in 0.25 s steps, so many trials miss by a quarter of a second. A faster 7 s timeout was tried and lowered delivery at 20% loss, so it was dropped.

6.2 Performance

Latency and throughput

Measurement	Result	Setting
SOS delivery latency, live devices	median 32 ms, p95 63 ms, max 78 ms	5 real processes, 30 SOS messages, all delivered
SOS delivery plus ACK back at the sender	median 47 ms, p95 87 ms	same 30 messages, all acknowledged
Latency under congestion, simulated	SOS median 540 ms against routine 2373 ms: 4.4 times faster	16 nodes, radio of one packet per 80 ms, 90 messages of which 15% are SOS, all to one gateway, 20 runs
API response time	median 16 ms, p95 31 ms	200 requests to a node's status

Throughput of one process

Simultaneous sends	All delivered	All acknowledged	Messages per second	Time to last ACK (s)
100	100	100	8413	0.02
500	500	500	7183	0.12
1000	1000	1000	7029	0.29
2000	2000	2000	4686	0.99

6.3 Reliability under packet loss

A 21-node mesh, 10 seeded runs per row, with the stated loss on every link. Delivery stays at 100% up to 10% loss and degrades gracefully; the extra transmissions per delivered message show the cost of retrying.

Loss per link	Delivery (mean)	Worst run	Transmissions per delivered	Runs
0%	100.0%	100.0%	5.8	10
10%	100.0%	100.0%	10.8	10
20%	98.8%	93.8%	16.1	10
30%	86.9%	68.8%	20.3	10
40%	77.0%	66.7%	26.5	10
50%	71.4%	57.1%	29.2	10

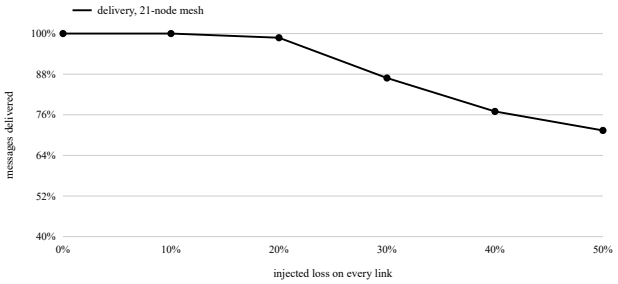


Figure 1. Delivery against injected loss.

6.4 Scalability

Grid meshes from 10 to 65 nodes at 5% and 20% loss, six runs per cell. Convergence is how long until every node knows every other node.

Nodes	Links	Delivery, 5% loss	Delivery, 20% loss	Convergence 5% (s)	Convergence 20% (s)
10	13	100.0%	100.0%	5.2	8.2
21	32	100.0%	98.8%	5.2	10.8
31	50	100.0%	96.2%	5.2	9.7
43	72	100.0%	96.2%	5.8	10.2
50	85	100.0%	92.9%	5.2	9.2
65	113	98.0%	83.1%	5.2	10.7

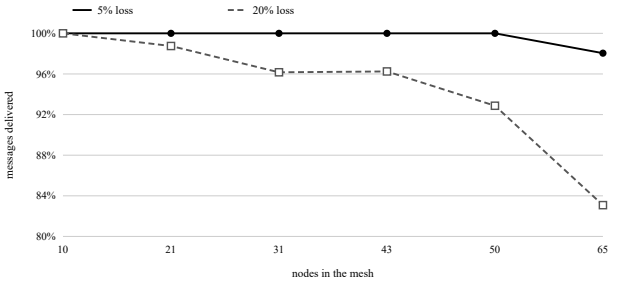


Figure 2. Delivery against mesh size.

6.5 Routing quality, recovery and availability

Weighted versus hop-count routing. The same two-path topology (a short route over a lossy link, a longer route over clean links), the same seeds and the same best-effort traffic, run once with each router, 20 runs each.

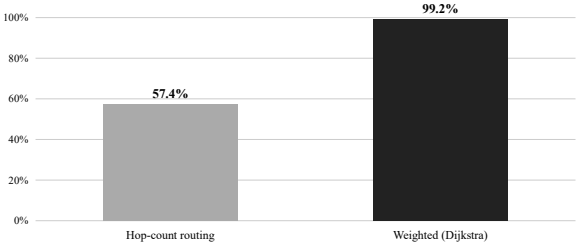


Figure 3. Delivery with each routing mode.

Failover and store-and-forward on real devices.

Trial type	Trials	Result
Kill a relay, then send	8	all delivered; reroute 3.5 to 4.4 s, mean 4.0 s
Gateway off, hold 5 SOS, switch on	4	all 5 delivered every time; flush 4.2 to 4.2 s, mean 4.2 s
Startup to full convergence	1	4.4 s for 5 processes
Simulated relay death	50	mean recovery 8.1 s (see K3)

All scenarios

Seven simulator scenarios, run with the specified timer values on a virtual clock (20 runs each, 50 for collapse_40).

Scenario	Nodes	Loss	Runs	Delivery (mean)	Worst run	Median relays	Convergence (s)
`baseline_10`	10	5%	20	100.0%	100.0%	1	5.5
`collapse_40`	40	20%	50	92.6%	78.6%	4	9.6
`no_gateway`	12	5%	20	100.0%	100.0%	2	5.2
`high_loss`	20	35%	20	89.3%	73.9%	2	17.7
`chain_10hop`	11	5%	20	100.0%	100.0%	9	14.4
`hopcount_vs_weighted`	5	45%	20	99.2%	90.0%	2	5.2
`priority_load`	16	0%	20	100.0%	100.0%	2	7.6

6.6 Security testing

Security is tested at three levels: unit tests of the cryptography (19 tests), node-level tests with encryption and signing on, and a live **attack drill** that runs three real nodes A, M and B with M hostile, and records what B does. The drill below was executed while this document was generated.

Attack	What M does	What B did	Result
Honest relay	M forwards A's message untouched.	B accepted and decrypted it	**Pass**
Tampered ciphertext (unsigned)	M flips one bit of the payload. Nothing but the GCM tag can notice.	B dropped it: drop_decrypt_failed	**Pass**
Tampered ciphertext (signed)	Same tampering, but A signed the message: the signature fails first.	B dropped it: drop_signature_invalid	**Pass**
Forged SOS	M signs a fake “SOS from A” with its own private key.	B dropped it: drop_signature_invalid	**Pass**

- A relay configured with no key at all still forwards correctly and never sees plaintext (`test_relay_never_sees_plaintext_payload`).
- A tampered ciphertext or a wrong key is detected by the GCM tag and dropped; the signature covers the ciphertext, so a signed message is rejected even earlier.
- A forged signature and an unknown signer are rejected and logged; signatures survive multi-hop relaying because only fields that never change in transit are signed.

> **Known limit.** Signing is opt-in across the mesh in this build, so a forger who leaves a packet unsigned is not caught by signature checks, and one pre-shared key protects the whole mesh. Both are listed as future work.

6.7 Resource use

Condition	Memory per process	CPU per process (share of one core)
Idle, 5 processes, 20 s	55 to 55 MB	at most 1.0%
Under a burst of traffic, 2.3 s	55 to 56 MB	at most 12.5%

The figures are for the Python process only, on one machine.

6.8 Usability, accessibility and compatibility

Aspect	What was checked	Outcome
Emergency usability	The SOS control is a single hold of about 1.2 s on one prominent button; releasing early sends nothing (DT-04); the need and location travel with it (DT-05); status is shown in plain words with the route (DT-06, DT-15, DT-16)	**Pass**

Aspect	What was checked	Outcome
Responsive layout	Both device screens at 390 px and 320 px wide (DT-17) and every page of the lab website at 390 px (UT-15): no sideways scrolling	**Pass**
Keyboard access	The SOS button is written to be armed with Space or Enter, but keyboard use and screen-reader use were not tested	**Not done**
Browsers	Google Chrome only (desktop and a phone-sized viewport). Firefox, Edge and Safari were not tested	**Partial**
Real phones	Not tested on physical devices; the PWA manifest and service worker are present but installation was not tested	**Partial**
Usability study	No timed study with participants was carried out; this is stated in the report as well	**Not done**

6.9 Maintainability

Every module has its own test file; the full suite runs in about 26 s with one command; measurement scripts re-create every table in the report from stored raw data, so a code change can be re-measured the same way.

7. Defects found and resolved

Every defect found while testing, with its cause and how it was fixed and retested. Defects D2, D3, D8 and D9 were found by testing the integrated system, not by the unit tests, which is why the integration, API and browser levels exist.

ID	Found by	Defect and cause	Fix and retest	Status
D1	Integration test of two components on one node	Only one of discovery and relay received packets: one receive callback, the last registrant won	A single dispatcher registered last; regression test added	**Closed**
D2	Phase IV integration	Multi-hop ACK and retry failed: duplicate filtering keyed on the message id dropped every ACK at the first relay and swallowed retries	Key on type, id and attempt; retransmission counter in the formerly reserved header byte; re-ACK duplicates; new tests that fail on the old code	**Closed**
D3	Phase IV integration	Dead nodes stayed routable: a newer link-state report was merged with the old one, so edges to a dead node never disappeared	A newer report replaces the origin's edges; tests added	**Closed**
D4	Fuzz test	A non-ASCII byte in a node-id field made the parser raise UnicodeDecodeError instead of PacketError, so one damaged packet could raise inside the receive path	Decode and encode failures now raise PacketError; both fuzz tests pass	**Closed**
D5	Repeated runs of the socket tests	Node.stop() hung in about a quarter of runs on Windows: wait_for swallowed a cancellation, so a background loop survived	Helper built on asyncio.wait; 15 of 15 test-file runs and 40 of 40 standalone iterations clean	**Closed**
D6	Repeated full-suite runs	Two timing-sensitive tests (test_api node-view hop distance; test_ack_multihop) failed in 2 and 1 of 6 runs under load; both pass alone	A wait_until polling helper replaces the fixed sleeps in both tests; the helpers have their own tests	**Closed**
D7	Repeated full-suite runs of the final suite	test_history_filters_by_priority_and_direction fails intermittently: two messages created in the same millisecond have no defined order, and the test expects newest first. The history query sorts by creation time only	Fixed at the cause: ties on creation time are now broken by row id, newest first (store.py). A deterministic regression test was added (test_messages_made_in_the_same_millisecond_list_newest_first); the original test passed 30 of 30 runs under CPU load	**Closed**
D8	Lab website test, first navigation click	The whole lab website went blank when a menu link was clicked: three React effects returned a value instead of nothing, which React then called as a cleanup function. Opening a page directly by URL did not show it	The effects now return nothing; the browser test (UT-01) clicks through every page and now passes	**Closed**
D9	Phone-width check of the lab website	The phone header's node selector ran past the right edge of the frame at 390 px width	Selector capped at 46% of the header width; UT-15 passes	**Closed**
D10	A fresh full-suite run during the final review	test_attack_drill_defends_every_attack failed about once in 15 runs, and under CPU load. The drill waited a fixed 0.6 s for the destination to learn Alice's public key and 0.3 s after each step. On a busy machine the key had not arrived yet, so a signed message was dropped as an unknown signer instead of an invalid signature, and the step was reported as not defended. No message was delivered in the failing runs; the verdict was wrong, not the defence	The drill now polls until the destination knows Alice's key and until the expected drop appears (up to 5 s), instead of sleeping. The test passed 30 of 30 runs under CPU load	**Closed**

8. Limitations and risks

- **No radio hardware.** The "radio" is a neighbour-restricted LAN or an in-memory simulator, so radio effects such as interference and range are modelled, not measured.
- **One machine.** All live-device timings were taken with five processes on one laptop. A two-laptop split worked over a phone hotspot, but it was not timed under the same protocol.
- **Recovery time is quantised.** Failure detection is defined by a 10 s heartbeat timer, so the "10 s or less" target is met only on average (K3).
- **Scale at high loss.** Delivery falls below 90% beyond about 43 nodes at 20% loss (K7).
- **Browser checks are not part of the pytest suite.** The two browser scripts are separate commands that need Chrome; the device script starts its own devices.
- **Untested:** a timed usability study, browsers other than Chrome, physical phones, installing the PWA.
- **No open defect.** The two intermittent tests found late (D7 and D10) were fixed at their cause and each passed 30 of 30 runs under CPU load.
- **Security scope.** One pre-shared key for the mesh; signing is optional; an unsigned forgery is not caught.

9. Conclusion and sign-off

EcoMesh was tested at six levels, from single-module unit tests to five real processes exchanging messages over TCP.

- **Functional:** 332 of 332 automated tests passed in the latest run, 18 of 18 browser checks of the device screens passed and 16 of 16 of the simulation lab. 64 of the 68 requirements are fully verified; 0 are partial, 2 were verified by inspection and 2 by a manual check.
- **Non-functional:** 8 of 10 target metrics are fully met. The 2 that are only partly met are K2 (convergence), K3 (recovery after relay death).
- **Design validated:** weighted routing delivered 99.2% against 57.4% for hop-count routing on identical seeds, SOS arrived 4.4 times faster under congestion, and every held message was flushed when a path returned.
- **Defects:** 10 found, 10 closed, none open.

The system is fit for its stated purpose, a software mesh demonstrated on LAN sockets and a simulator, with the limits in Section 8 stated openly.

Sign-off

Role	Name	Signature	Date
Prepared by	Tanvi Sheth (2024300239)		
Prepared by	Twissha Shah (2024300231)		
Prepared by	Miti Shah (2024300230)		
Reviewed by (guide)	Prof. Anuj Tawari		

Appendix A: all 332 automated test cases

Every automated test case, grouped by file. The three symbols show the three earlier repeated runs (P = passed, F = failed); the last column is the latest run.

`test_packet.py` *47 tests · Wire format: 52-byte header, path, payload, signature, retransmission counter, malformed input*

#	Test case	Earlier runs	Latest
1	test_a_retransmission_keeps_its_signable_identity	PPP	**Pass**
2	test_attempt_defaults_to_zero_and_is_range_checked	PPP	**Pass**
3	test_attempt_round_trips_through_the_reserved_byte	PPP	**Pass**
4	test_bad_magic_rejected	PPP	**Pass**
5	test_default_ttl_is_10	PPP	**Pass**
6	test_flag_bits_independent	PPP	**Pass**
7	test_header_is_52_bytes	PPP	**Pass**
8	test_hop_count_defaults_to_zero	PPP	**Pass**
9	test_identity_bytes_changes_if_msg_id_changes	PPP	**Pass**
10	test_identity_bytes_changes_if_payload_changes	PPP	**Pass**
11	test_identity_bytes_changes_if_src_or_dst_changes	PPP	**Pass**
12	test_identity_bytes_reflects_flags_as_currently_set	PPP	**Pass**
13	test_identity_bytes_survives_a_relay_hop_unchanged	PPP	**Pass**
14	test_magic_bytes_are_M_E	PPP	**Pass**
15	test_msg_id_defaults_to_uuid4_and_round_trips	PPP	**Pass**
16	test_msg_id_is_16_bytes_on_wire	PPP	**Pass**
17	test_node_id_longer_than_8_bytes_rejected	PPP	**Pass**
18	test_path_len_field_matches_path	PPP	**Pass**
19	test_path_too_long_rejected	PPP	**Pass**
20	test_payload_len_field_matches_payload	PPP	**Pass**
21	test_payload_too_large_rejected	PPP	**Pass**
22	test_remaining_length_includes_signature	PPP	**Pass**
23	test_remaining_length_matches_actual_body_size	PPP	**Pass**
24	test_remaining_length_rejects_bad_magic	PPP	**Pass**
25	test_remaining_length_rejects_wrong_size_input	PPP	**Pass**
26	test_reserved_byte_is_zero	PPP	**Pass**
27	test_round_trip_all_message_types_and_priorities	PPP	**Pass**
28	test_round_trip_no_path_no_payload	PPP	**Pass**
29	test_round_trip_with_path_and_payload	PPP	**Pass**
30	test_round_trip_with_signature	PPP	**Pass**
31	test_signable_bytes_changes_if_path_changes	PPP	**Pass**
32	test_signable_bytes_excludes_payload_and_signature	PPP	**Pass**
33	test_signable_bytes_matches_pack_header_and_path_region	PPP	**Pass**
34	test_signable_bytes_reflects_flags_as_currently_set	PPP	**Pass**
35	test_signable_bytes_validates_like_pack	PPP	**Pass**
36	test_signature_present_implies_signed_flag_even_if_not_set_explicitly	PPP	**Pass**
37	test_signed_flag_without_signature_rejected	PPP	**Pass**
38	test_src_dst_full_8_char_ids	PPP	**Pass**
39	test_src_dst_null_padded_to_8_bytes	PPP	**Pass**
40	test_timestamp_is_uint64_milliseconds	PPP	**Pass**
41	test_truncated_header_rejected	PPP	**Pass**

#	Test case	Earlier runs	Latest
42	test_truncated_path_rejected	PPP	**Pass**
43	test_truncated_payload_rejected	PPP	**Pass**
44	test_truncated_signature_rejected	PPP	**Pass**
45	test_ttl_out_of_range_rejected	PPP	**Pass**
46	test_version_field	PPP	**Pass**
47	test_wrong_signature_length_rejected	PPP	**Pass**

test_link_state.py *33 tests · Link-state encoding, gossip, re-flooding, report replacement, convergence*

#	Test case	Earlier runs	Latest
48	test_a_dead_node_stops_being_reachable_once_its_neighbours_stop_reporting_it	PPP	**Pass**
49	test_a_newer_report_removes_links_the_origin_no_longer_has	PPP	**Pass**
50	test_dated_report_round_trip	PPP	**Pass**
51	test_decode_dated_report_rejects_too_short_payload	PPP	**Pass**
52	test_decode_empty_payload	PPP	**Pass**
53	test_decode_rejects_truncated_payload	PPP	**Pass**
54	test_encode_caps_failures_at_255	PPP	**Pass**
55	test_encode_decode_round_trip	PPP	**Pass**
56	test_forget_neighbour_removes_from_graph_and_own_report	PPP	**Pass**
57	test_no_public_key_decodes_as_none	PPP	**Pass**
58	test_non_link_state_packet_ignored	PPP	**Pass**
59	test_on_topology_change_fires_when_graph_actually_changes	PPP	**Pass**
60	test_own_report_looped_back_is_ignored	PPP	**Pass**
61	test_public_key_round_trips	PPP	**Pass**
62	test_reachable_from_follows_edge_direction	PPP	**Pass**
63	test_receiving_report_merges_reported_edges_into_graph	PPP	**Pass**
64	test_receiving_report_teaches_learned_public_key	PPP	**Pass**
65	test_receiving_report_teaches_learned_role	PPP	**Pass**
66	test_record_failure_invalidates_cached_routes	PPP	**Pass**
67	test_record_failure_updates_graph_immediately_not_just_on_next_report	PPP	**Pass**
68	test_record_link_quality_updates_graph_immediately	PPP	**Pass**
69	test_report_is_reflooded_to_other_neighbours	PPP	**Pass**
70	test_report_reflooded_reaches_a_node_two_hops_away	PPP	**Pass**
71	test_role_round_trips_for_all_roles	PPP	**Pass**
72	test_send_report_includes_public_key_from_provider	PPP	**Pass**
73	test_send_report_includes_role_from_role_provider	PPP	**Pass**
74	test_send_report_reaches_each_neighbour	PPP	**Pass**
75	test_send_report_syncs_own_direct_edges_into_graph	PPP	**Pass**
76	test_send_report_uses_recorded_quality_and_failures	PPP	**Pass**
77	test_stale_own_edge_removed_when_neighbour_drops_out	PPP	**Pass**
78	test_stale_report_is_not_reflooded_or_remerged	PPP	**Pass**
79	test_two_nodes_converge_on_each_others_graph_view	PPP	**Pass**
80	test_unknown_public_key_does_not_fire_on_public_key_learned	PPP	**Pass**

test_node.py *28 tests · Whole node: discovery + relay + routing + reliability + security + logging together*

#	Test case	Earlier runs	Latest
81	test_ack_never_arriving_exhausts_retries_and_queues_for_store_forward	PPP	**Pass**

#	Test case	Earlier runs	Latest
82	test_discovery_and_relay_both_work_on_the_same_node	PPP	**Pass**
83	test_encrypted_payload_decrypts_correctly_at_destination	PPP	**Pass**
84	test_forged_signature_is_rejected_not_delivered	PPP	**Pass**
85	test_held_message_is_sent_the_moment_its_neighbour_appears	PPP	**Pass**
86	test_message_to_discovered_neighbour_delivers_without_manual_next_hop	PPP	**Pass**
87	test_middle_node_auto_relays_across_a_chain	PPP	**Pass**
88	test_neighbours_property_matches_discovery_table	PPP	**Pass**
89	test_no_key_configured_drops_encrypted_message	PPP	**Pass**
90	test_node_originates_a_message_three_hops_away_via_reflooded_gossip	PPP	**Pass**
91	test_relay_never_sees_plaintext_payload	PPP	**Pass**
92	test_rescue_dst_routes_to_gateway_across_multiple_hops	PPP	**Pass**
93	test_route_forgotten_when_neighbour_dies	PPP	**Pass**
94	test_router_forwards_beyond_direct_neighbours_via_gossip	PPP	**Pass**
95	test_send_with_ack_delivers_and_gets_acknowledged	PPP	**Pass**
96	test_send_without_route_queues_to_store_forward	PPP	**Pass**
97	test_signature_survives_multi_hop_relay_and_verifies_at_destination	PPP	**Pass**
98	test_signed_message_verifies_and_delivers_one_hop	PPP	**Pass**
99	test_store_forward_evicts_lowest_priority_when_queue_is_full	PPP	**Pass**
100	test_store_forward_expires_messages_past_max_age	PPP	**Pass**
101	test_store_forward_flushes_once_a_route_appears	PPP	**Pass**
102	test_store_logs_drop_reason	PPP	**Pass**
103	test_store_logs_forward_on_middle_node	PPP	**Pass**
104	test_store_logs_peer_up_and_down	PPP	**Pass**
105	test_store_logs_sent_message_and_delivery_lifecycle	PPP	**Pass**
106	test_unknown_signer_is_rejected	PPP	**Pass**
107	test_unsigned_messages_still_work_when_no_keypair_configured	PPP	**Pass**
108	test_wrong_session_key_is_rejected_not_delivered	PPP	**Pass**

`test_store.py` *27 tests · SQLite message, event and peer log and its queries*

#	Test case	Earlier runs	Latest
109	test_all_peers	PPP	**Pass**
110	test_context_manager_closes_connection	PPP	**Pass**
111	test_count_events	PPP	**Pass**
112	test_events_after_limit_keeps_the_newest	PPP	**Pass**
113	test_events_after_returns_only_newer_events_oldest_first	PPP	**Pass**
114	test_events_by_kind	PPP	**Pass**
115	test_events_for_message_ordered_by_time	PPP	**Pass**
116	test_events_since_returns_only_newer_events_in_order	PPP	**Pass**
117	test_get_unknown_message_returns_none	PPP	**Pass**
118	test_get_unknown_peer_returns_none	PPP	**Pass**
119	test_list_and_count_messages_filter_by_direction_and_status	PPP	**Pass**
120	test_list_messages_filters_by_priority	PPP	**Pass**
121	test_messages_by_status	PPP	**Pass**
122	test_messages_made_in_the_same_millisecond_list_newest_first	PPP	**Pass**
123	test_public_key_round_trips_as_blob	PPP	**Pass**
124	test_record_and_get_message	PPP	**Pass**
125	test_record_event_and_query_by_message	PPP	**Pass**

#	Test case	Earlier runs	Latest
126	test_record_event_returns_row_id	PPP	**Pass**
127	test_record_message_stores_path_as_comma_separated	PPP	**Pass**
128	test_record_message_upserts_on_same_id	PPP	**Pass**
129	test_remove_peer	PPP	**Pass**
130	test_update_message_route_records_hops_and_path	PPP	**Pass**
131	test_update_message_status	PPP	**Pass**
132	test_update_status_of_unknown_message_returns_false	PPP	**Pass**
133	test_upsert_and_get_peer	PPP	**Pass**
134	test_upsert_peer_preserves_link_quality_when_not_given	PPP	**Pass**
135	test_upsert_peer_updates_role_and_last_seen	PPP	**Pass**

test_ack_retry.py *20 tests · ACK tracker, retry limits, store-and-forward bounds, eviction and expiry*

#	Test case	Earlier runs	Latest
136	test_acknowledge_before_timeout_clears_pending	PPP	**Pass**
137	test_acknowledge_unknown_msg_id_returns_false	PPP	**Pass**
138	test_default_max_age_matches_design_of_algorithms_t_hold	PPP	**Pass**
139	test_default_max_count_matches_design_of_algorithms_c_max	PPP	**Pass**
140	test_enqueue_and_all	PPP	**Pass**
141	test_enqueue_over_the_bound_evicts_lowest_priority	PPP	**Pass**
142	test_enqueue_same_msg_id_twice_does_not_duplicate	PPP	**Pass**
143	test_enqueue_under_the_bound_evicts_nothing	PPP	**Pass**
144	test_eviction_never_displaces_sos_for_routine_traffic	PPP	**Pass**
145	test_eviction_ties_broken_by_oldest_first	PPP	**Pass**
146	test_exhausting_max_retries_reports_failed_and_stops_tracking	PPP	**Pass**
147	test_expire_applies_regardless_of_priority	PPP	**Pass**
148	test_expire_removes_messages_older_than_max_age	PPP	**Pass**
149	test_expire_returns_empty_when_nothing_is_stale	PPP	**Pass**
150	test_multiple_pending_sends_tracked_independently	PPP	**Pass**
151	test_multiple_queued_messages_independent	PPP	**Pass**
152	test_no_timeout_before_deadline	PPP	**Pass**
153	test_remove_clears_entry	PPP	**Pass**
154	test_remove_unknown_id_is_a_no_op	PPP	**Pass**
155	test_timeout_triggers_retry_and_resets_timer	PPP	**Pass**

test_crypto.py *19 tests · AES-256-GCM encryption and Ed25519 signatures, wrong key, tampering*

#	Test case	Earlier runs	Latest
156	test_associated_data_matching_succeeds	PPP	**Pass**
157	test_associated_data_must_match_on_decrypt	PPP	**Pass**
158	test_ciphertext_does_not_contain_plaintext	PPP	**Pass**
159	test_ciphertext_is_nonce_prefixed_per_section_7_2	PPP	**Pass**
160	test_decrypt_rejects_input_shorter_than_nonce	PPP	**Pass**
161	test_decrypt_with_wrong_key_fails	PPP	**Pass**
162	test_encrypt_decrypt_round_trip	PPP	**Pass**
163	test_encrypt_rejects_wrong_key_size	PPP	**Pass**
164	test_generate_keypair_public_bytes_is_32	PPP	**Pass**
165	test_generate_session_key_is_32_bytes	PPP	**Pass**
166	test_load_public_key_rejects_wrong_size	PPP	**Pass**

#	Test case	Earlier runs	Latest
167	test_load_public_key_round_trip	PPP	**Pass**
168	test_sign_verify_round_trip	PPP	**Pass**
169	test_tampered_ciphertext_fails_gcm_tag_check	PPP	**Pass**
170	test_two_encryptions_of_same_plaintext_differ	PPP	**Pass**
171	test_verify_fails_for_tampered_message	PPP	**Pass**
172	test_verify_fails_for_wrong_public_key_forged_sos	PPP	**Pass**
173	test_verify_never_raises_on_garbage_signature	PPP	**Pass**
174	test_verify_rejects_wrong_length_signature	PPP	**Pass**

test_reliability.py *18 tests · Duplicate suppression and the priority queue*

#	Test case	Earlier runs	Latest
175	test_all_three_priorities_ordered_correctly	PPP	**Pass**
176	test_contains	PPP	**Pass**
177	test_different_messages_are_independent	PPP	**Pass**
178	test_entry_expires_after_window	PPP	**Pass**
179	test_entry_still_duplicate_just_inside_window	PPP	**Pass**
180	test_first_sighting_is_not_a_duplicate	PPP	**Pass**
181	test_len_and_bool	PPP	**Pass**
182	test_len_reflects_current_entries_after_prune	PPP	**Pass**
183	test_ordering_uses_packet_priority_even_with_custom_item	PPP	**Pass**
184	test_peek_does_not_remove	PPP	**Pass**
185	test_peek_empty_queue_returns_none	PPP	**Pass**
186	test_push_with_item_returns_item_not_packet	PPP	**Pass**
187	test_push_without_item_defaults_to_packet_itself	PPP	**Pass**
188	test_same_priority_and_timestamp_does_not_crash_on_packet_comparison	PPP	**Pass**
189	test_second_sighting_is_a_duplicate	PPP	**Pass**
190	test_sos_pops_before_normal_regardless_of_insertion_order	PPP	**Pass**
191	test_sos_reapplied_mid_route_overtakes_already_queued_normal	PPP	**Pass**
192	test_ties_broken_by_oldest_timestamp_first	PPP	**Pass**

test_relay.py *18 tests · Forwarding, TTL, loop detection, priority at a relay, dedup rules*

#	Test case	Earlier runs	Latest
193	test_a_retransmission_is_forwarded_by_a_relay_that_saw_the_original	PPP	**Pass**
194	test_an_ack_is_not_a_duplicate_of_the_data_it_acknowledges	PPP	**Pass**
195	test_delivers_packet_addressed_to_self	PPP	**Pass**
196	test_different_messages_not_treated_as_duplicates	PPP	**Pass**
197	test_drops_on_loop_detected	PPP	**Pass**
198	test_drops_on_send_failure	PPP	**Pass**
199	test_drops_when_no_route_configured	PPP	**Pass**
200	test_drops_when_ttl_expired	PPP	**Pass**
201	test_duplicate_delivery_is_dropped	PPP	**Pass**
202	test_duplicate_forward_is_dropped_not_resent	PPP	**Pass**
203	test_forwards_to_configured_next_hop	PPP	**Pass**
204	test_ignores_hello_packets	PPP	**Pass**
205	test_no_dedup_configured_allows_reprocessing	PPP	**Pass**
206	test_on_forward_hook_called_with_next_hop	PPP	**Pass**
207	test_single_forward_still_sent_immediately	PPP	**Pass**

#	Test case	Earlier runs	Latest
208	test_sos_forwarded_before_already_queued_normal	PPP	**Pass**
209	test_the_destination_sees_a_retried_message_once_but_is_told_about_it	PPP	**Pass**
210	test_two_hop_relay_a_to_b_to_c	PPP	**Pass**

test_transport_sim.py *15 tests · Simulated radio: links, loss, latency, delivery*

#	Test case	Earlier runs	Latest
211	test_broadcast_delivers_only_to_linked_neighbours	PPP	**Pass**
212	test_broadcast_with_no_links_does_nothing	PPP	**Pass**
213	test_link_config_rejects_invalid_latency_range	PPP	**Pass**
214	test_link_config_rejects_loss_out_of_range	PPP	**Pass**
215	test_loss_probability_one_always_drops	PPP	**Pass**
216	test_loss_probability_zero_never_drops	PPP	**Pass**
217	test_neighbours_of_reflects_link_matrix	PPP	**Pass**
218	test_remove_link	PPP	**Pass**
219	test_seeded_network_reproduces_same_loss_pattern	PPP	**Pass**
220	test_send_delivers_with_zero_latency	PPP	**Pass**
221	test_send_respects_latency_delay	PPP	**Pass**
222	test_send_without_link_raises_transport_error	PPP	**Pass**
223	test_set_link_can_be_unidirectional	PPP	**Pass**
224	test_set_link_is_bidirectional_by_default	PPP	**Pass**
225	test_stop_unregisters_from_network	PPP	**Pass**

test_discovery.py *15 tests · HELLO beacons, neighbour table, death detection, range emulation*

#	Test case	Earlier runs	Latest
226	test_get_and_snapshot	PPP	**Pass**
227	test_hello_includes_public_key_when_configured	PPP	**Pass**
228	test_hello_is_broadcast_on_start	PPP	**Pass**
229	test_hello_omits_public_key_when_not_configured	PPP	**Pass**
230	test_neighbour_added_on_hello_received	PPP	**Pass**
231	test_neighbour_marked_dead_after_timeout	PPP	**Pass**
232	test_neighbour_public_key_is_none_when_peer_has_none	PPP	**Pass**
233	test_neighbour_table_learns_public_key_from_hello	PPP	**Pass**
234	test_non_hello_packet_ignored	PPP	**Pass**
235	test_on_peer_up_fires_once_for_new_neighbour	PPP	**Pass**
236	test_own_hello_not_added_as_neighbour	PPP	**Pass**
237	test_prune_removes_only_stale_neighbours	PPP	**Pass**
238	test_stop_cancels_background_tasks	PPP	**Pass**
239	test_update_reports_new_neighbour_once	PPP	**Pass**
240	test_visible_neighbours_filters_unlisted_senders	PPP	**Pass**

test_routing.py *14 tests · Cost function and weighted path selection*

#	Test case	Earlier runs	Latest
241	test_add_node_creates_empty_entry	PPP	**Pass**
242	test_edge_cost_matches_section_8_2_formula	PPP	**Pass**
243	test_edge_cost_penalises_recent_failures	PPP	**Pass**
244	test_edge_cost_penalises_weak_link	PPP	**Pass**
245	test_edge_cost_perfect_link_is_cheapest	PPP	**Pass**

#	Test case	Earlier runs	Latest
246	test_edge_cost_rejects_negative_failures	PPP	**Pass**
247	test_edge_cost_rejects_out_of_range_quality	PPP	**Pass**
248	test_len_counts_nodes	PPP	**Pass**
249	test_nodes_lists_every_known_node	PPP	**Pass**
250	test_remove_edge	PPP	**Pass**
251	test_remove_node_drops_incoming_and_outgoing_edges	PPP	**Pass**
252	test_set_edge_creates_both_endpoints	PPP	**Pass**
253	test_set_edge_is_directed_not_automatically_reversed	PPP	**Pass**
254	test_set_edge_overwrites_existing	PPP	**Pass**

test_dijkstra.py *14 tests · Shortest path, RESCUE__ resolution, route cache*

#	Test case	Earlier runs	Latest
255	test_equal_quality_prefers_fewer_hops	PPP	**Pass**
256	test_normal_node_role_not_treated_as_terminal	PPP	**Pass**
257	test_path_to_self_has_no_next_hop	PPP	**Pass**
258	test_recent_failures_also_penalise_a_link	PPP	**Pass**
259	test_rescue_dst_none_when_no_gateway_reachable	PPP	**Pass**
260	test_rescue_dst_picks_cheapest_of_multiple_gateways	PPP	**Pass**
261	test_rescue_dst_resolves_to_nearest_gateway	PPP	**Pass**
262	test_route_is_cached_until_invalidated	PPP	**Pass**
263	test_self_as_gateway_resolves_trivially	PPP	**Pass**
264	test_set_role_invalidates_cache	PPP	**Pass**
265	test_shortest_path_direct_neighbour	PPP	**Pass**
266	test_shortest_path_multi_hop_chain	PPP	**Pass**
267	test_unreachable_destination_returns_none	PPP	**Pass**
268	test_weighted_routing_avoids_lossy_shortcut	PPP	**Pass**

test_link_quality.py *12 tests · Link quality measured from HELLO reception; hop-count routing mode*

#	Test case	Earlier runs	Latest
269	test_a_flapping_link_keeps_its_history_instead_of_resetting_to_perfect	PPP	**Pass**
270	test_a_long_silence_starts_the_estimate_afresh	PPP	**Pass**
271	test_a_new_link_is_not_penalised_for_hellos_not_yet_due	PPP	**Pass**
272	test_a_returning_lossy_neighbour_is_reported_even_if_it_was_already_reported_before	PPP	**Pass**
273	test_forgetting_a_neighbour_resets_its_history	PPP	**Pass**
274	test_hop_count_routing_takes_the_flaky_short_path	PPP	**Pass**
275	test_losing_every_other_hello_scores_about_half	PPP	**Pass**
276	test_node_exposes_the_hop_count_switch	PPP	**Pass**
277	test_perfect_reception_scores_one	PPP	**Pass**
278	test_quality_change_is_reported_only_when_it_moves_enough	PPP	**Pass**
279	test_unknown_neighbour_has_no_link_quality	PPP	**Pass**
280	test_weighted_routing_avoids_the_flaky_short_path	PPP	**Pass**

test_api.py *12 tests · REST and WebSocket API against a live mesh (front end to engine)*

#	Test case	Earlier runs	Latest
281	test_attack_drill_defends_every_attack	PPP	**Pass**
282	test_bad_requests_get_clear_errors	PPP	**Pass**

#	Test case	Earlier runs	Latest
283	test_killing_a_node_reroutes_and_reviving_restores_it	PPP	**Pass**
284	test_node_view_reports_peers_with_hop_distance	PPP	**Pass**
285	test_presets_and_scenarios_are_listed	PPP	**Pass**
286	test_protocol_inspector_builds_and_parses_real_packets	PPP	**Pass**
287	test_reset_swaps_the_preset_and_security_options	PPP	**Pass**
288	test_route_compare_returns_both_routers	PPP	**Pass**
289	test_security_playground_round_trips_and_detects_tampering	PPP	**Pass**
290	test_snapshot_describes_a_converged_mesh	PPP	**Pass**
291	test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered	PPP	**Pass**
292	test_websocket_streams_a_hello_snapshot_then_events	PPP	**Pass**

test_robustness.py *11 tests · Fuzzing, boundary input, concurrency and stress*

#	Test case	Earlier runs	Latest
293	test_bit_flipped_valid_packets_never_crash_the_parser	PPP	**Pass**
294	test_empty_payload_is_a_valid_message	PPP	**Pass**
295	test_header_only_input_and_empty_input_are_rejected	PPP	**Pass**
296	test_node_id_longer_than_eight_bytes_is_rejected	PPP	**Pass**
297	test_node_survives_garbage_arriving_on_the_wire	PPP	**Pass**
298	test_payload_over_the_length_field_is_rejected	PPP	**Pass**
299	test_random_bytes_never_crash_the_parser	PPP	**Pass**
300	test_replayed_packet_is_delivered_only_once	PPP	**Pass**
301	test_send_to_an_unknown_destination_is_held_not_raised	PPP	**Pass**
302	test_sos_burst_and_routine_burst_from_two_senders_do_not_corrupt_each_other	PPP	**Pass**
303	test_two_hundred_simultaneous_sends_across_two_hops_all_arrive_exactly_once	PPP	**Pass**

test_lan_transport.py *6 tests · Real TCP transport between processes, range emulation, three-node mesh*

#	Test case	Earlier runs	Latest
304	test_full_node_mesh_over_real_tcp_discovers_routes_and_delivers_with_ack	PPP	**Pass**
305	test_receiver_sees_the_previous_hop_not_the_packet_source	PPP	**Pass**
306	test_sending_to_a_dead_peer_fails_fast_with_transport_error	PPP	**Pass**
307	test_sending_to_a_non_neighbour_is_refused	PPP	**Pass**
308	test_switched_off_device_neither_sends_nor_hears	PPP	**Pass**
309	test_unknown_previous_hop_and_garbage_are_ignored_and_listener_survives	PPP	**Pass**

test_sim_runner.py *5 tests · Simulator runner and topologies*

#	Test case	Earlier runs	Latest
310	test_build_chain_topology_links_only_adjacent_nodes	PPP	**Pass**
311	test_build_full_mesh_topology_links_every_pair	PPP	**Pass**
312	test_chain_topology_discovers_only_direct_neighbours	PPP	**Pass**
313	test_full_mesh_topology_discovers_all_other_nodes	PPP	**Pass**
314	test_visible_neighbours_map_restricts_discovery_within_a_linked_topology	PPP	**Pass**

test_ack_multihop.py *3 tests · ACK over several hops, lost-ACK re-send, retry through relays*

#	Test case	Earlier runs	Latest
315	test_a_lost_ack_is_re_sent_when_the_sender_retries	PPP	**Pass**
316	test_a_retry_crosses_relays_that_already_forwarded_the_original	PPP	**Pass**
317	test_ack_returns_over_the_path_the_data_took	PPP	**Pass**

test_device_api.py *12 tests · The device server: events, send, history filters, structured SOS, I am safe, CSV export, rescue reply*

#	Test case	Earlier runs	Latest
318	test_a_safe_check_in_never_overtakes_an_sos	PPP	**Pass**
319	test_device_learns_a_multi_hop_route_to_the_rescue_team	PPP	**Pass**
320	test_events_show_the_neighbour_coming_up_and_since_filters	PPP	**Pass**
321	test_history_filters_by_priority_and_direction	PPP	**Pass**
322	test_lan_ip_is_a_usable_ipv4_address	PPP	**Pass**
323	test_payload_parsing_handles_structured_and_plain_text	PPP	**Pass**
324	test_rescue_exports_received_requests_as_csv	PPP	**Pass**
325	test_rescue_replies_to_the_survivor_across_the_mesh	PPP	**Pass**
326	test_safe_check_in_reaches_rescue_at_status_priority	PPP	**Pass**
327	test_send_delivers_a_text_message_across_the_mesh	PPP	**Pass**
328	test_send_rejects_bad_requests	PPP	**Pass**
329	test_structured_sos_round_trip_and_plain_sos_still_works	PPP	**Pass**

test_conftest_helpers.py *3 tests · The wait_until polling helpers used by the timing-sensitive tests*

#	Test case	Earlier runs	Latest
330	test_async_wait_until_accepts_coroutine_conditions	PPP	**Pass**
331	test_wait_until_fails_clearly_after_the_timeout	PPP	**Pass**
332	test_wait_until_returns_the_first_truthy_value	PPP	**Pass**